

pack の padx/pady と configure の padx/pady の違い

はい、完全に別物です。名前は同じですが、それぞれ「余白を作る場所」も「設定できるウィジェットの範囲」も異なります。

決定的な違い:余白を作る「場所」が違う

```
entry.pack(padx=10)          # ウィジェットの外側に余白を作る
button.configure(padx=10)    # ウィジェットの内側(中身と枠の間)に余白を作る
```

図にすると、以下のようなイメージです。



pack(padx=...): 配置マネージャーが管理する「外側の余白」

これは `pack()` という配置の仕組み側が持っているオプションで、「このウィジェットの周りに、これだけの隙間を空けて配置する」という指定です。

```
button1.pack(side="left", padx=10)
button2.pack(side="left", padx=10)
# → ボタン同士の間隔が20px(両方のpadxが合算される)空く
```

特徴: `pack`, `grid`, `place` のどの配置方法を使っても、どんなウィジェットに対しても共通で使えます(`Entry`, `Text`, `Listbox`など、`configure`側で`padx`が使えないウィジェットでも問題なく使える)。

configure(padx=...): ウィジェット自身が持つ「内側の余白」

こちらはウィジェット自体の描画に関するオプションで、「ウィジェットの枠と、中の文字(や画像)との間に、これだけの隙間を空ける」という指定です。

```
button = tk.Button(root, text="OK")
button.configure(padx=20, pady=10)
# → ボタンの枠が、文字の左右に20px, 上下に10pxの余白を持つ分だけ大きくなる
```

特徴: ウィジェットの種類によって対応の有無が分かります(Button, Labelは対応、Entry, Listboxは非対応)。

具体例で比較

```
import tkinter as tk

root = tk.Tk()

# pack側のpadx: ボタンの「外側」に余白 → ボタン自体の見た目(大きさ)は変わらない
button1 = tk.Button(root, text="外側余白")
button1.pack(padx=30)

# configure側のpadx: ボタンの「内側」に余白 → ボタン自体が大きくなる
button2 = tk.Button(root, text="内側余白")
button2.configure(padx=30)
button2.pack()

root.mainloop()
```

button1 は見た目のサイズはそのまま、周囲にスペースができます。button2 はボタン自体が横に大きく広がって表示されます。画面上の見た目としては全く違う結果になります。

まとめ

	pack(padx=...)	configure(padx=...)
余白の場所	ウィジェットの外側(周囲の空間)	ウィジェットの内側(枠と中身の間)
見た目への影響	ウィジェット自体の大きさは変わらない	ウィジェット自体が大きくなる
使えるウィジェット	全てのウィジェットで共通して使える	ウィジェットの種類によって対応状況が異なる(Entryは非対応など)
設定する場所	配置時(pack()/grid())の引数	ウィジェット作成時 or configure()

同じ「padx」という名前が使われているため紛らわしいですが、「配置(pack)の余白」と「ウィジェット自体(configure)の余白」という、全く別の階層の設定だと理解しておくでと混乱しにくいです。